

SUSPICIOUS POWERSHELL ACTIVITY DETECTION

Simulating, Investigating, and Detecting Encoded PowerShell Activity
in a Windows Active Directory Lab

Ryan Morales

Active Directory Home Lab

Splunk Enterprise | Sysmon | PowerShell | Windows 10

August 2026

1. Project Overview

The goal of this project was to simulate suspicious PowerShell activity, investigate the resulting events in Splunk, and develop a completed detection rule that could be converted into a scheduled alert. On WS01, suspicious-looking activity was simulated by executing a harmless PowerShell command using the -EncodedCommand parameter and a Base64-encoded string.

The resulting Sysmon Event ID 1 process creation activity was investigated in Splunk using fields including Image, ParentImage, and CommandLine. The CommandLine field showed the use of -EncodedCommand, while the process relationship showed PowerShell launching another PowerShell process. The use of -EncodedCommand or its abbreviated -enc form was determined to be suspicious enough to warrant further investigation, but not sufficient on its own to determine that the activity was malicious.

A Splunk detection was then developed to identify PowerShell process creation events on WS01 containing either -EncodedCommand or -enc within the command line. The completed detection was converted into a scheduled alert and successfully validated using fresh PowerShell activity, confirming that Splunk could automatically identify and alert on the simulated behavior.

2. Lab Environment

System / Tool	Role
WS01 - 192.168.100.20	Served as the Windows workstation where the simulated suspicious PowerShell activity was generated.
DC01 - 192.168.100.10	Served as the domain controller and hosted Splunk Enterprise, where telemetry from WS01 was investigated and analyzed.
Sysmon	Generated process creation telemetry on WS01. Sysmon Event ID 1 provided information about the PowerShell processes, including Image, CommandLine, and ParentImage.
Splunk Universal Forwarder	Forwarded telemetry collected on WS01 to Splunk Enterprise running on DC01.
PowerShell	Used on WS01 to generate the harmless encoded PowerShell activity used during the simulation and validation.
Splunk Enterprise	Used to investigate process creation telemetry, develop the PowerShell detection, configure the scheduled alert, and validate that the alert triggered successfully.

3. Attack Simulation

Suspicious-looking PowerShell activity was generated on WS01 using a harmless command that returned the message PowerShell Detection Test. The command was converted into a Base64-encoded string and then executed using PowerShell's -EncodedCommand parameter.

Base64 encoding was used to simulate PowerShell activity in which the underlying command is less immediately readable. Although encoded PowerShell can be used to obscure malicious instructions, the use of -EncodedCommand does not automatically indicate malicious activity and can also have legitimate uses.

When the encoded command was executed on WS01, PowerShell successfully decoded and executed the command and returned PowerShell Detection Test as the output. The command itself was harmless, but its execution generated the suspicious process creation telemetry needed for investigation and detection development.

```
Windows PowerShell
PS C:\WINDOWS\system32> $encoded
VwByAGkAdABlAC9AbwB1AHQAABlAHQAIAAIAFAAbwB3AGUAcgBTAGgAZQBzAGwAIABEAGUAdABlAGMAdABvAGkAbgAgAFQAZQBzAHQAIAgA=
PS C:\WINDOWS\system32> powershell.exe -EncodedCommand $encoded
PowerShell Detectoin Test
PS C:\WINDOWS\system32> |
```

Figure 1 - Encoded PowerShell activity generated on WS01 using the -EncodedCommand parameter and a Base64-encoded command.

4. Splunk Investigation

After generating the encoded PowerShell activity, Sysmon Event ID 1 events from WS01 were investigated in Splunk. Event ID 1 represents process creation and provides information that can be used to understand what process executed, how it executed, and what process was responsible for launching it.

The Image field identified what executable ran, while the CommandLine field showed how the executable was launched. In the simulated event, the CommandLine contained the -EncodedCommand parameter followed by a Base64-encoded string. This behavior was considered suspicious enough to warrant further investigation because the encoded string made the underlying PowerShell instructions less immediately readable.

The ParentImage field identified the process responsible for launching the new PowerShell process. For the simulated activity, the parent process was also powershell.exe. This was expected because the encoded PowerShell process had been launched from an existing PowerShell session on WS01.

< Hide Fields		All Fields	Format	Show: 20 Per Page	View: List
a CurrentDirectory 1		a Description 1			
a EventCode 1		a EventType 1			
a FileVersion 1		a Hashes 1			
a Image 1		a Index 1			
a IntegrityLevel 1		a Keywords 1			
a Linecount 1		a LogName 1			
a LogonGuid 1		a LogonId 1			
a Message 1		a OpCode 1			
a OriginalFileName 1		a ParentCommandLine 1			
a ParentImage 1		a ParentProcessGuid 1			
a ParentProcessId 1		a ParentUser 1			
a ProcessGuid 1		a ProcessId 1			
a Product 1		a RecordNumber 1			
a RuleName 1		a Sd 1			
a SdType 1		a SourceName 1			
a Splunk_server 1		a TaskCategory 1			
a TerminalSessionId 1		a Type 1			
a User 2		a UtcTime 1			
+ Extract New Fields					
		Type	Field	Value	Action
Selected		☒	host	WS01	☒
		☒	source	WinEventLog:Microsoft-Windows-Sysmon/Operational	☒
		☒	sourcetype	WinEventLog:Microsoft-Windows-Sysmon/Operational	☒
Event		☒	CommandLine	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VwByAGkAdABlAC9AbwB1AHQAABlAHQAIAAIAFAAbwB3AGUAcgBTAGgAZQBzAGwAIABEAGUAdABlAGMAdABvAGkAbgAgAFQAZQBzAHQAIAgA=	☒
		☐	Company	Microsoft Corporation	☒
		☐	ComputerName	WS01.home.local	☒
		☐	CurrentDirectory	C:\WINDOWS\system32\	☒
		☐	Description	Windows PowerShell	☒
		☐	EventCode	1	☒
		☐	EventType	4	☒
		☐	FileVersion	10.0.26100.8875 (WinBuild.160101.0800)	☒
		☐	Hashes	MD5=E65ECC3754249C00F50A035202F7591SHA256=7600FFE12DA44FE89D035B13801E8E91D064BC544A27B19A5CF49F6A88B18F5JMPHASH=AFACF6DC904114B198160AAB4D0AE77	☒
		☐	Image	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	☒
		☐	IntegrityLevel	Medium	☒
		☐	Keywords	None	☒
		☐	LogName	Microsoft-Windows-Sysmon/Operational	☒
		☐	LogonGuid	{78c37862-8037-6a90-a4fb-0b0000000000}	☒
		☐	LogonId	0xBFBA4	☒
		☐	Message	Process Create: RuleName: - UtcTime: 2026-08-27 18:46:32.133 ProcessGuid: {78c37862-8608-6a90-9502-000000000000} ProcessId: 3644 Image: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe FileVersion: 10.0.26100.8875 (WinBuild.160101.0800) Description: Windows PowerShell Product: Microsoft® Windows® Operating System Company: Microsoft Corporation OriginalFileName: PowerShell.EXE CommandLine: "C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VwByAGkAdABlAC9AbwB1AHQAABlAHQAIAAIAFAAbwB3AGUAcgBTAGgAZQBzAGwAIABEAGUAdABlAGMAdABvAGkAbgAgAFQAZQBzAHQAIAgA= CurrentDirectory: C:\WINDOWS\system32\ User: HOME\ryan.admin LogonGuid: {78c37862-8037-6a90-a4fb-0b0000000000} LogonId: 0xBFBA4 TerminalSessionId: 1 IntegrityLevel: Medium Hashes: MD5=E65ECC3754249C00F50A035202F7591SHA256=7600FFE12DA44FE89D035B13801E8E91D064BC544A27B19A5CF49F6A88B18F5JMPHASH=AFACF6DC904114B198160AAB4D0AE77 ParentProcessGuid: {78c37862-8076-6a90-3a01-000000000000} ParentProcessId: 6072 ParentImage: C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe ParentCommandLine: "C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" ParentUser: HOME\ryan.admin	☒
		☐	OpCode	Info	☒
		☐	OriginalFileName	PowerShell.EXE	☒
		☐	ParentCommandLine	"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe"	☒
		☐	ParentImage	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	☒

Figure 2 - Sysmon Event ID 1 in Splunk showing the encoded PowerShell process, including the Image, ParentImage, and CommandLine fields used during the investigation.

The presence of -EncodedCommand was not considered proof of malicious activity because encoded PowerShell commands can also be used for legitimate purposes. Instead, the Base64 string recorded in the Splunk event was copied and decoded separately using PowerShell on DC01. Decoding the artifact revealed the harmless command used during the simulation: Write-Output "PowerShell Detection Test".

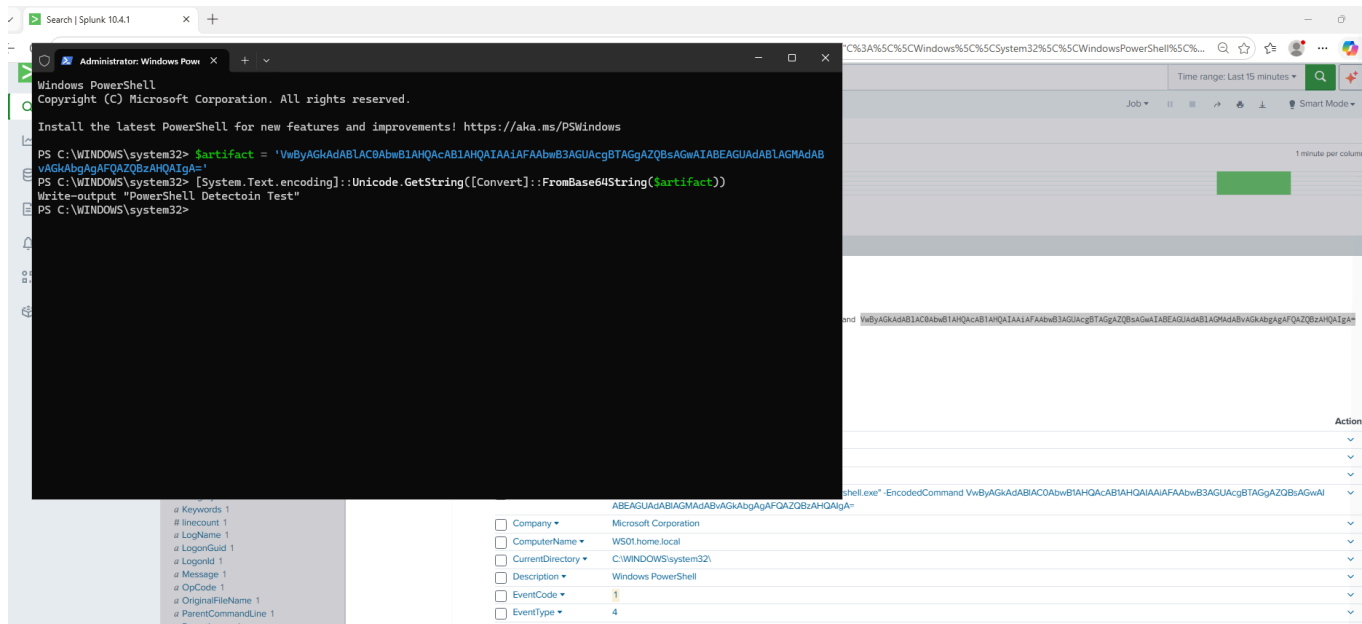


Figure 3 - Base64 artifact extracted from the PowerShell telemetry and decoded separately on DC01 to reveal the underlying command.

5. Detection Development

After investigating the simulated activity, a Splunk detection was developed to identify similar encoded PowerShell process creation events on WS01. The search was limited to host=WS01 because the workstation was the source of the process telemetry, while EventCode=1 limited the results to Sysmon process creation events.

The Image field was filtered using Image="*powershell.exe" to focus the detection on PowerShell process creation. The CommandLine field was then searched for either the full -EncodedCommand parameter or its abbreviated -enc form. The OR condition allowed either form to satisfy the detection logic and improved coverage by preventing the search from relying only on the full parameter.

The completed SPL detection was:

```
index=main host=WS01 EventCode=1
Image="*powershell.exe"
(CommandLine="*-EncodedCommand *" OR CommandLine="*-enc *")
| table _time User Image ParentImage CommandLine
```

The final table command organized matching events into columns containing _time, User, Image, ParentImage, and CommandLine. These fields provided an analyst with the time of the event, the user responsible for the process, what executable ran, what process launched it, and how the executable was executed.

The abbreviated -enc condition was separately validated by generating another encoded PowerShell process using -enc instead of -EncodedCommand. The resulting event successfully matched the detection and displayed -enc within the CommandLine field, confirming that both variations were detected.

Time	User	Image	ParentImage	CommandLine
2025-08-27 14:11:48.477	NOT_TRANSLATED HOME\ryan.adm	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VbYAGkAD8LAC8dwB1AHQACAB1AHQAIAMAFABwB3AGUAcgBTAGaZQ8sAGwI4BEAGUASB1AGMADABvAGkAbgAFQAZQ8z
2025-08-27 13:46:32.145	NOT_TRANSLATED HOME\ryan.adm	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VbYAGkAD8LAC8dwB1AHQACAB1AHQAIAMAFABwB3AGUAcgBTAGaZQ8sAGwI4BEAGUASB1AGMADABvAGkAbgAFQAZQ8z
2025-08-27 15:16:08.122	NOT_TRANSLATED HOME\ryan.adm	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -EncodedCommand VbYAGkAD8LAC8dwB1AHQACAB1AHQAIAMAFABwB3AGUAcgBTAGaZQ8sAGwI4BEAGUASB1AGMADABvAGkAbgAFQAZQ8z
2025-08-27 15:13:18.428	NOT_TRANSLATED HOME\ryan.adm	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe	"C:\WINDOWS\System32\WindowsPowerShell\v1.0\powershell.exe" -enc VbYAGkAD8LAC8dwB1AHQACAB1AHQAIAMAFABwB3AGUAcgBTAGaZQ8sAGwI4BEAGUASB1AGMADABvAGkAbgAFQAZQ8z

Figure 4 - Completed SPL detection showing encoded PowerShell events and successful coverage of both -EncodedCommand and the abbreviated -enc parameter.

6. Alert Configuration

The completed detection was configured as a scheduled Splunk alert named Suspicious PowerShell Encoded Command. The alert was scheduled to run every five minutes and search the previous ten minutes of telemetry.

The ten-minute lookback provided overlap between scheduled searches and helped reduce the possibility of missing relevant activity because of telemetry ingestion delays, forwarding delays, or events occurring near a search boundary. If the detection returned one or more matching events during the search window, the alert would trigger.

The alert was assigned a severity of High and configured to appear within Splunk's Triggered Alerts. Converting the completed SPL search into a scheduled alert automated the detection process rather than requiring the search to be manually executed each time suspicious PowerShell activity needed to be identified.

Save As Alert

×

Settings

Title

Suspicious PowerShell Encoded Command

Description

Optional

Permissions

Private

Shared in App

Alert type

Scheduled

Real-time

Run on Cron Schedule ▾

Time Range

Last 10 minutes ▸

Cron Expression

*/5 * * * *

e.g. 00 18 *** (every day at 6PM).

[Learn More](#)

Expires

24

hour(s) ▾

Trigger Conditions

Trigger alert when

Number of Results ▾

is greater than ▾

0

Trigger

Once

For each result

Throttle ?

☐

Trigger Actions

Cancel

Save

Figure 5 - Splunk alert configuration showing the scheduled five-minute execution, ten-minute lookback, and result-based trigger condition.

7. Detection Results

To perform the final validation, a fresh -EncodedCommand PowerShell event was generated on WS01 using the same harmless simulation method used earlier in the project. The validation command returned the message PowerShell Alert Validation.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> $command = 'Write-Output "PowerShell Alert Validation"'
PS C:\WINDOWS\system32> $bytes = [System.Text.Encoding]::Unicode.GetBytes($command)
PS C:\WINDOWS\system32> $encoded = [Convert]::ToBase64String($bytes)
PS C:\WINDOWS\system32> $encoded
VwByAGkAdABlAC8ATwB1AHQAAB1AHQAIAA1AFAAbwB3AGUAcgBTAGgAZQBsAGWAIABBAGwAZQByAHQAIBWAGEAbABpAQQAYQB8AGkAbwBuACIA
PS C:\WINDOWS\system32> powershell.exe -EncodedCommand $encoded
PowerShell Alert Validation
PS C:\WINDOWS\system32>

```

Figure 6 - Fresh encoded PowerShell activity generated on WS01 for final scheduled-alert validation.

After generating the activity, the SPL detection was not manually executed. Instead, Splunk's Triggered Alerts section was checked to determine whether the scheduled alert would automatically identify the new activity. The Suspicious PowerShell Encoded Command alert successfully triggered at approximately 3:05 PM following the simulated activity.

The screenshot shows the Splunk Enterprise interface for Triggered Alerts. The alert list contains one entry:

Time	Alert name	App	Type	Severity	Mode	Actions
2026-08-28 15:05:07 CDT	Suspicious PowerShell Encoded Command	search	Scheduled	High	Digest	View Results Edit Alert Delete

Figure 7 - High-severity Suspicious PowerShell Encoded Command alert successfully displayed in Splunk Triggered Alerts.

The triggered alert was opened for further investigation. The matching event displayed the fields selected by the detection's table command, including `_time`, `User`, `Image`, `ParentImage`, and `CommandLine`. These fields corresponded with the newly generated PowerShell activity on WS01 and confirmed that the expected event had triggered the alert.

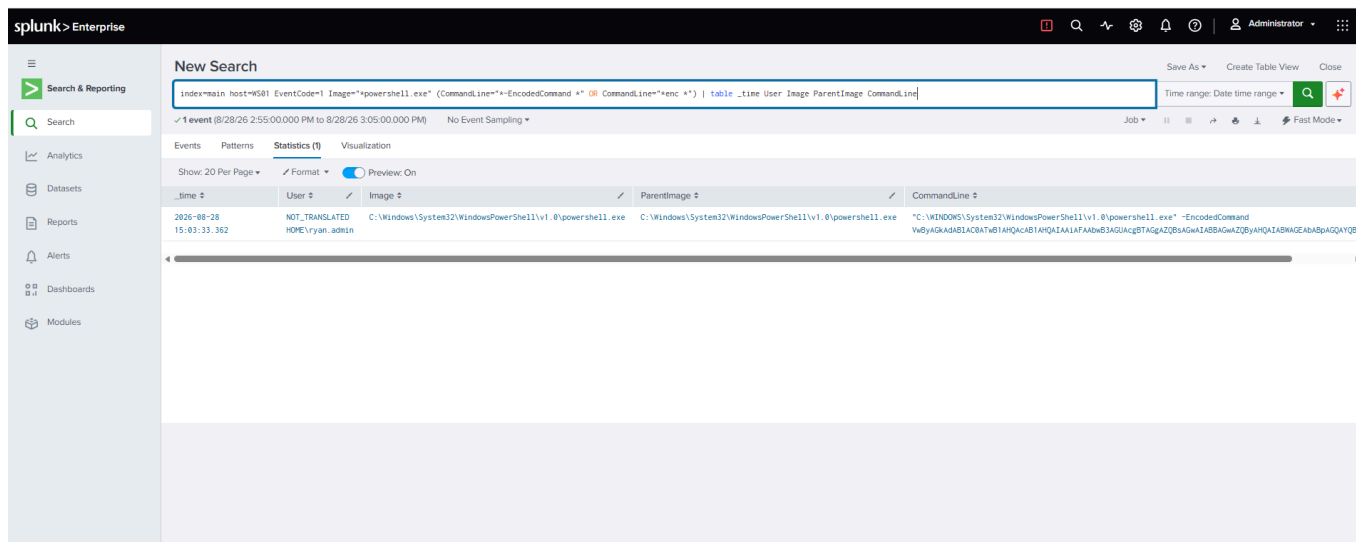


Figure 8 - Triggered alert investigation showing the matching PowerShell event and the analyst-focused fields returned by the detection.

Previous testing had already demonstrated that the SPL detection could manually identify encoded PowerShell activity. This final validation went further by confirming that the complete detection workflow functioned as intended. The activity was generated on WS01, forwarded into Splunk, identified by the scheduled detection, and automatically surfaced as a High-severity triggered alert without requiring the SPL search to be manually executed.

8. Skills Demonstrated & Lessons Learned

This project gave me hands-on experience with log analysis, SPL development, and detection tuning to narrow large amounts of telemetry into events that are relevant for investigation. I also gained practical experience investigating Sysmon process creation telemetry and using the resulting information to develop and validate a behavioral detection in Splunk.

Before completing this project, I did not have hands-on experience investigating Sysmon Event ID 1. I now have a much stronger understanding of process creation events and several of their important fields. The Image field identifies what executable ran, ParentImage provides context about the process responsible for launching it, and CommandLine shows how the process was executed and what arguments or parameters were supplied.

The project also reinforced the difference between suspicious and malicious activity. Seeing powershell.exe execute with -EncodedCommand is behavior that warrants investigation, but it does not automatically indicate that a workstation has been compromised. Encoded PowerShell can have legitimate uses, meaning additional context and analysis are necessary before reaching a conclusion.

I also practiced investigating the activity beyond the initial detection. The Base64 artifact captured within the CommandLine field was extracted from Splunk and decoded separately from the analyst side of the lab. This demonstrated how an analyst can move from identifying suspicious behavior to examining the underlying command and determining what the activity was actually attempting to execute.

If I encountered similar activity as a SOC analyst, I would treat the encoded PowerShell command as a reason to investigate further rather than immediately classify it as malicious. I would examine the affected workstation and user, review the Image, ParentImage, and full CommandLine, decode the Base64 content, and use the resulting context to determine whether the activity represented legitimate PowerShell usage or a potential security incident.